Dealer Pre-Install Setup SOP

Version: 1.1 — 2026-07-28 **Audience**: Dealer shop staff preparing hardware for customer installs **Goal**: Ship hardware to a customer's home such that the install crew can plug in and have a working system within ~15 minutes, instead of doing full setup on-site.

When to use this guide

Use this checklist whenever you have **a confirmed install scheduled** with the customer's information in hand. Pre-config is one-shot per controller/bridge — re-pre-configuring takes the same time as the first pass.

If the customer's information is uncertain (especially WiFi password and SSID spelling), **stop and confirm before proceeding**. Re-keying creds on-site is faster than driving back to fix a pre-config error.

What you'll need at the workbench

- WLED controller (Skikibily 4-channel, Dig-Octa 8-channel, or Generic per install plan)
- ESP32 bridge — already flashed with Lumina v1.2 firmware (see Appendix A if not)
- 12V power supply for the controller (matched to LED power requirements; bench supply is fine)
- Micro-USB or USB-C cable to power the bridge (5V from any USB-A port)
- Laptop or phone with WiFi
- Customer info sheet — **completed before opening this guide** (see Section 1)
- Label printer or fine-tip Sharpie
- Sticker stock or masking tape

Section 1 — Information to capture from the customer (BEFORE you start)

This is the most important step. Confirm every field with the customer and read it back to them.

```
Customer Info Sheet — Install # _____

Customer name:                _____________________________
Customer email (Lumina acct): _____________________________
Phone:                        _____________________________
Install address:              _____________________________
Install date:                 _____________________________

WiFi
```

```
  SSID (exact case + spelling): _____________________________
  Password:                     _____________________________
  Band (2.4 GHz required):      [ ] 2.4 GHz confirmed
  Hidden network?               [ ] yes  [ ] no
  Guest network?                [ ] yes  [ ] no  (avoid — usually blocks LAN device-to-device)

Network details (ask if customer knows; otherwise skip — DHCP is fine):
  Router brand/model:           _____________________________
  IP assignment preference:     [ ] DHCP (default)
                                [ ] Static IP — _______________
  Subnet:                       _________________ (e.g. 192.168.1.0/24)
  Gateway:                      _________________

Controller hardware (per install plan / site survey):
  Controller type:              [ ] Dig-Octa (8ch)
                                [ ] Skikibily (4ch)
                                [ ] Generic WLED
  LED type:                     [ ] SK6812 RGBW (default — type 30)
                                [ ] WS2814 RGBW (type 30)
                                [ ] WS2812B RGB
                                [ ] Other: ____________________
  Color order:                  [ ] RGB (default - order 1)  [ ] other: ____
  Channel plan:
    Ch 0 — pin ___ — _____ LEDs — name: ___________________
    Ch 1 — pin ___ — _____ LEDs — name: ___________________
    Ch 2 — pin ___ — _____ LEDs — name: ___________________
    Ch 3 — pin ___ — _____ LEDs — name: ___________________
    Ch 4 — pin ___ — _____ LEDs — name: ___________________
    Ch 5 — pin ___ — _____ LEDs — name: ___________________
    Ch 6 — pin ___ — _____ LEDs — name: ___________________
    Ch 7 — pin ___ — _____ LEDs — name: ___________________
  Total LED count:              _____ LEDs
  Max power draw (calc):        _____ W @ 12V  (0.3W per LED @ full white as rough estimate)
```

Trust the QuinLED Dig-Octa channel-to-GPIO map (Section 9 / Appendix B). The board's designers validated GPIO 0, 1, 2, 3 for LED data despite their strapping/UART role. If WLED's UI flags them red, that warning is expected and safe to dismiss on Dig-Octa hardware. The canonical map is: LED 1=GPIO 0, LED 2=GPIO 1, LED 3=GPIO 2, LED 4=GPIO 3, LED 5=GPIO 4, LED 6=GPIO 5, LED 7=GPIO 13, LED 8=GPIO 21.

Section 2 — WLED controller pre-config

2.0 Flash the PINNED WLED version FIRST (REQUIRED — do not skip)

Flashing method is right; "flash the latest" is wrong. The standing rule is to flash via **quinled.info** — correct. But quinled.info's web flasher serves **the LATEST** WLED release, which today is **0.15.4**. A dealer who flashes "latest" to the letter produces a **stalling controller with no error surfaced anywhere** — it passes every other step in this SOP and ships broken.

Pin the version per board:

Board / variant	WLED version to flash	Notes
SKIKBILY 4-channel (ESP32_Ethernet)	**0.15.1**	What the vendor ships and what is field-proven
Dig-Octa 8-channel	0.15.1	Same family; pin the same until re-qualified

Why 0.15.1 specifically (two independent reasons):

1. **A confirmed 0.15.4 stall regression on this exact board.** Field-measured on two same-variant SKIKBILY 4-channel controllers (both `ESP32_Ethernet`), with the WLED version as the *only* differing variable:
   - **0.15.1** (vendor factory firmware) → **69–87 fps, rock-steady, no stall.**
   - **0.15.4** (reflashed latest) → 170 fps between stalls, but **drops to 0 fps for 4–6s every ~~17s~~ (30% duty cycle)**, a both-core pause (HTTP goes unreachable during the stall too). Ruled out by live data: realtime (`live:false`), memory (heap flat), reboot (uptime monotonic), AudioReactive (off), WiFi (rssi −30s, bssid fixed), the app (force-stopped, still stalls), the bridge (unplugged), and other LAN nodes (none). Only the version differs.
2. **The Lumina app's effect catalog is pinned to 0.15.1 fx IDs** (`lib/features/patterns/canonical_palettes.dart`, `design_models.dart` — the `WledEffectsCatalog` verified against a 0.15.1 device). A controller on a different WLED build risks effect-ID drift, so the app would apply the wrong effect. Matching the firmware to the catalog keeps effect selection correct.

After flashing, VERIFY the version (one command — makes this self-detecting):

```
curl -s http://4.3.2.1/json/info | python3 -c "import sys,json;d=json.load(sys.stdin);print('release',d.get('release')
# SKIKBILY 4-channel MUST read:  release ESP32_Ethernet | ver 0.15.1
```

Confirm BOTH: `release` matches the board variant AND `ver` matches the pin. If `ver` reads 0.15.4 (or anything but the pin), re-flash the pinned version before continuing — every later step will otherwise "pass" on a controller that stalls.

Treat this as a standing never-default of the flash procedure, same class as *erase flash first*, *ABL never default*, *disable AudioReactive* (§2.5), and *verify NTP actually synced*.

2.1 Bench power up

1. Connect the controller to its 12V power supply (no LED strip needed yet — controller boots without LEDs attached).
2. Wait ~30 seconds. The controller boots into WLED AP mode if no WiFi creds are saved.
3. On your laptop or phone WiFi list, look for `WLED-AP` (or similar — the SSID may include the chip's MAC suffix).
4. Connect to it. Default password is `wled1234`.
5. The captive portal page should auto-open at `http://4.3.2.1`. If not, open a browser and navigate there manually.

2.2 Save the customer's WiFi credentials

1. In the WLED captive portal, click **WiFi Settings**.
2. Set **Network name (SSID)**: enter exactly what's on the customer info sheet (case-sensitive).
3. Set **Network password**: copy exactly. **Avoid extra spaces.**
4. Set **WLED status name** (mDNS hostname): use a customer-recognizable prefix, e.g. `lumina-<customer-last-name>` or `lumina-<install-num>`. This will appear as `<name>.local` on the customer's LAN.
5. Set **AP Mode** options:
   - **Always serve AP** → OFF
   - **AP password** → `wled1234` (keep default; only matters if WiFi connect fails at the install site)
6. (Optional) If the customer needs a **static IP**:
   - Toggle **Static IP** ON
   - Enter the assigned IP, subnet (255.255.255.0 usually), and gateway from the customer info sheet
   - Note: most installs use DHCP. Static IP is only needed if the customer has multiple WLED controllers or wants port-forwarded direct access.
7. Click **Save & Connect**.
8. The controller will reboot and try to connect to the customer's WiFi. **It will fail** because the customer's WiFi isn't reachable at your shop. After ~30 seconds, the controller falls back to AP mode. **This is expected and harmless** — the credentials are saved to flash and will be used when the controller boots at the install site.

2.3 Configure LED hardware

1. Reconnect to the controller's AP (it should be back in AP mode after the failed WiFi connect).
2. Navigate to `http://4.3.2.1` → **Config** → **LED Preferences**.
3. **Hardware setup** section:
   - **LED count (total)**: enter total from customer info sheet.
   - **Max current**: set to a safe value below your power supply rating. A reasonable default: `LED count × 60mA`. For 188 LEDs at 60mA full white → ~11A. **Set max current to the lower of (PSU rating) and (calculated)**. WLED will dim automatically if it exceeds this — protects the wire and PSU.
   - **Auto-calculate brightness limit** → ON.
4. **LED outputs** section — add one bus per active channel. Click **+ LED output** to add each:
   - **Type**: SK6812 RGBW (type 30) unless customer info says otherwise.
   - **Color order**: **RGB (order 1)** unless customer info says otherwise. WLED appends the white channel automatically on an RGBW bus type. This is the same value the app's Nex-Gen Standard push writes (`type: 30`, `order: 1`) — do not "fix" it to GRB.
   - **Start index**: cumulative — first bus starts at 0, second bus starts at first bus's count, etc.
   - **Count**: from customer info sheet, per channel.
   - **Pin**: from the canonical channel-to-GPIO map below. Do **not** rely on the old "avoid GPIO 0/3/12" rule of thumb — it was wrong. GPIO 12 isn't exposed as an LED channel on the Dig-Octa at all, and GPIO 0/1/2/3 are validated for LED data on this board even though WLED's UI flags them red.

QuinLED Dig-Octa Brainboard-32-8L — canonical map:

Channel	ESP32 GPIO	Notes
LED 1	GPIO 0	Strapping pin — works in production
LED 2	GPIO 1	UART TX0 — serial logging unavailable while in use
LED 3	GPIO 2	Strapping pin
LED 4	GPIO 3	UART RX0
LED 5	GPIO 4	General-purpose, reliable
LED 6	GPIO 5	General-purpose, reliable
LED 7	GPIO 13	General-purpose, reliable
LED 8	GPIO 21	Shares the I2C SCL bus — avoid pairing with I2C peripherals

   - **Reverse**: leave unchecked unless install plan calls for reversed direction.
   - **Skip first LEDs**: 0 unless there's a known dead/dummy LED at the start.

5. **Defaults** section:
   - **Apply preset**: leave 0 (no startup preset — the app will set the on/off state).
   - **Turn LEDs on after power up** → OFF (avoid surprise blast of light if the customer's lights re-power before the app connects). Or leave ON if customer prefers immediate "on" behavior — note their preference on the sheet.
6. **Brightness limiter**: set to ~85% as a safety margin. Customer can raise later from the app.
7. Click **Save**.
8. Click **Reboot** at the bottom of the Config page to apply hardware changes cleanly.

2.3b Time, timezone, and location (REQUIRED — do not skip)

A controller whose clock never syncs fires NO schedules at all. This is the single highest-impact silent failure we ship: everything else works, the app reports success, and the customer's lights simply never come on by themselves. Set it at the bench.

1. Navigate to `http://4.3.2.1` → **Config** → **Time & Macros**.
2. **Get time from NTP server** → **ON**.
3. **NTP server host** → `time.google.com`. The stock pool host fails on some customer networks; this is a standing never-default.
4. **Time Zone** → the customer's local zone. A controller left on UTC fires every wall-clock timer shifted by the UTC offset.
5. **Latitude / Longitude** → the install site's coordinates. Leaving them at `0,0` computes sunrise/sunset for the Gulf of Guinea.

6. Click **Save**.

Verification (on-site, once the controller has internet): open the controller's Info page and confirm the displayed time is correct local wall-clock time. A date in 1970 (or 2106) means NTP never synced — the clock is unset and no timer will fire. The app also surfaces this: the customer's **Schedule** tab shows a red banner with a **How to fix** action, and the installer wizard's handoff **pre-flight check** shows a *Controller clock* row.

> **Why the coordinates matter:** sunrise/sunset scheduling is live, and the app **refuses to arm a solar timer** when latitude/longitude are unset or left at `0,0`. Skip this and the customer's "on at sunset" schedule silently never arms. Note the system supports **one sunrise and one sunset schedule** (positional timer slots), and offsets have no editor field yet.

2.4 Configure segments to match channels

This step makes the Lumina app's channel-aware features work correctly.

1. After reboot, reconnect to the controller's AP.
2. Navigate to `http://4.3.2.1` → main page → click the **Segments** icon (3 horizontal bars).
3. By default, WLED creates one segment covering all LEDs. **Delete it** and add one segment per channel:
   - **Segment 0**: Start = 0, Stop = (Ch 0 LED count). Name: `Ch 0 – Front Roofline` (or whatever the customer-facing name is from the sheet).
   - **Segment 1**: Start = (Ch 0 count), Stop = (Ch 0 + Ch 1 count). Name: per sheet.
   - Continue for each active channel.
4. **Important**: segment Start/Stop indices must exactly match the bus boundaries from Section 2.3, otherwise channel filtering in the Lumina app won't behave correctly.
5. Save.

2.5 Disable the AudioReactive usermod (REQUIRED — do not skip)

The flash image ships this usermod ENABLED with a digital mic pre-configured on GPIO 32/15 — pins our hardware does not have — so disable it. Our controllers flashed with the AudioReactive build variant (name ends in `-AR`, e.g. `Dig-Octa-ESP32-8L-Eth-AR`) build the usermod default-on. Its idle I2S/FFT path is needless overhead on a mic-less unit, so turning it off is correct housekeeping.

> **Correction (field finding):** AudioReactive was *originally* blamed for the periodic effect freeze, on the strength of a short symptom-free window after a manual disable. That was an over-conclusion — the freeze is **intermittent**, so a brief clean window proved nothing. The freeze was later traced to the **WLED 0.15.4 stall regression (see §2.0)**, reproduced with AudioReactive already OFF. So: still disable it (overhead), but it is **not** the freeze cause — the version pin in §2.0 is.

Treat this as a standing never-default of the flash procedure, in the same class as *erase flash first*, *ABL never default*, *pin the WLED version* (§2.0), and *NTP host* → `time.google.com`.

1. Navigate to `http://4.3.2.1` → **Config** → **Usermods**.
2. Find the **AudioReactive** section.
3. Set **Enable** → **OFF**.
4. Click **Save**.
5. **No reboot needed** — the controller suspends audio processing immediately. Do **not** change the mic type or GPIO fields while you're in there; those are the only usermod settings that *would* require a reboot, and leaving them alone keeps the change clean.

Verify (optional, from a laptop on the controller's AP):

```
curl http://4.3.2.1/json/cfg
# → "um":{"AudioReactive":{"enabled":false, ...}}
```

> **Already-shipped controllers:** the Lumina app self-heals this. Its controller-defaults healer reads `um.AudioReactive.enabled` on each local connect and surgically disables it (no reboot) when it finds it on. That covers the existing fleet as owners open the app on-site — but it only works on the local network, so **still do this step at the bench** rather than leaving a customer to see a frozen effect first.

> **If a controller genuinely has a microphone** (a unit built for Audio Mode), disabling this turns Audio Mode off — and the app's healer will keep turning it off on every connect. No controller shipped to date has a working mic, so this step is correct for all current builds. Raise it with engineering before flashing any mic-equipped unit.

2.6 Smoke-test on bench (optional but recommended)

If you have a spare LED strip on the bench, plug it into channel 0 and verify it lights when you tap a color on the WLED UI. Disconnect after — the controller ships without strips attached.

2.7 Label the controller

Print or write on a label affixed to the controller body:

```
Lumina Install # _____
Customer: _____
WLED SSID: _____
WLED name: lumina-<customer>.local
Channels: X channels, Y total LEDs
Pre-configured: 2026-MM-DD by <staff initials>
```

Keep this label readable — the install crew uses it as the source-of-truth confirmation when they unbox on-site.

2.8 Power down

Disconnect from the controller's AP. Power off the 12V supply. The controller's saved WiFi creds and LED config persist in flash across power cycles.

Section 3 — ESP32 bridge pre-config

3.1 Power up and connect to the bridge AP

1. Plug the bridge into a USB power source (laptop USB-A or a USB wall adapter).
2. Wait ~30 seconds. The bridge boots and starts WiFiManager AP mode (because NVS is empty after the most recent firmware flash).
3. On your laptop or phone WiFi list, look for `Lumina-XXXX` (last 4 of the bridge's MAC). The MAC was logged when the bridge was flashed (see Appendix A) — match it to the bridge you're holding.
4. Connect to that AP. **No password required** (open AP).
5. The captive portal page should auto-open. If not, navigate to `http://192.168.4.1`.

3.2 Save the customer's WiFi credentials

1. In the WiFiManager captive portal, click **Configure WiFi**.
2. The bridge will scan for nearby networks. **Your shop's WiFi will appear in the list** but the customer's won't (they're not in range). **Don't click on a listed network** — instead manually enter:
    - **SSID**: customer's SSID from info sheet (case-sensitive, exact).
    - **Password**: customer's password.
3. Click **Save**.
4. The bridge will reboot, attempt to connect to the customer's WiFi, fail at your shop, fall back to AP. **This is expected.** Credentials are saved to NVS.

3.3 Verify the saved credentials (optional sanity check)

If you have a phone-based hotspot you can name to the customer's exact SSID with the customer's password, you can verify the bridge connects:

1. On your phone, enable Personal Hotspot.
2. Rename the hotspot SSID to match the customer's SSID exactly.
3. Set the hotspot password to match the customer's password exactly.
4. Power-cycle the bridge.
5. Within ~30 seconds, the bridge should connect to your hotspot (it can't tell the difference between your hotspot pretending to be the customer's network and the real network).
6. Confirm by checking your hotspot's connected-clients list — should show the bridge.
7. Power-cycle bridge and shut off hotspot when done.

This verification step is optional but recommended for high-stakes installs.

3.4 Label the bridge

Print or write on a label affixed to the bridge body:

```
Lumina Install # _____________
Customer: ____________________
Bridge deviceId: <MAC-no-colons, e.g. D4E9F4FA8E78>
Bridge AP name: Lumina-<last-4-of-MAC>
Pre-configured: 2026-MM-DD by <staff initials>
```

The bridge `deviceId` is the MAC address with colons stripped. It's printed during firmware flash and visible in WiFiManager's AP name. Record it on the label so the install crew can match it in the Lumina app's bridge picker.

3.5 Power down

Unplug the bridge. NVS persists; on next power-up at the customer's home it'll auto-connect to their WiFi.

Section 4 — Pairing strategy

You have two options. **Default to Option A** unless customer pickup/shipping logistics specifically require Option B.

Option A — Pair on-site (recommended)

- Skip pre-pairing.
- At the install site, after the bridge powers up and connects to customer WiFi, open the Lumina app on the customer's phone or your install tablet (signed in as the customer).
- Bridge appears in the app's bridge-discovery flow within ~1 minute.
- Tap **Pair** in the app → bridge accepts the pendingUid within 5 seconds → status flips to `paired`.
- Total time: ~2 minutes.

This is faster, simpler, and avoids the "wrong WiFi at pairing time" failure modes below.

Option B — Pre-pair at the shop (advanced)

Only use if you must ship a fully-paired bridge to a customer without an install visit.

1. Set up a phone hotspot named **exactly** like the customer's WiFi (same SSID + password) so the bridge can connect at the shop.
2. Power on the bridge — it connects to your hotspot, signs in to Firebase, registers in `/bridge_registry/<deviceId>`.
3. On a laptop or staff device, sign in to the Lumina app as the customer (you need their email + password OR a delegated staff account with permissions on their UID).
4. Trigger the bridge-pair flow in the app. The app writes `pendingUid` to the registry doc.

5. Within 5s the bridge sees `pendingUid`, persists the customer UID to NVS, sets status `paired`.
6. Power down the bridge. Shut off the hotspot.
7. Bridge is now paired. WiFi creds in NVS remain set to the customer's SSID — that's correct.

Note: WiFi creds are tied to the SSID/password, not to a specific hotspot. As long as the customer's home WiFi has the same SSID and password the bridge was paired against, it'll connect cleanly on power-up at the install site.

Section 5 — On-site quick-deploy checklist (install day)

By the time the crew arrives, the controller and bridge are pre-configured. The on-site sequence:

```
[ ] 1. Mount controller in its planned location. Connect LED strips to the
       pre-labeled channels. Connect 12V PSU. Power on.
[ ] 2. Verify controller's status LED behavior is normal (per WLED docs:
       solid = connected; blink patterns = various error states).
[ ] 3. Plug bridge into a 5V USB power adapter near the controller (or
       wherever the customer wants it — a closet shelf is fine; it just
       needs WiFi reach to the home network).
[ ] 4. Wait ~30s. Bridge connects to customer WiFi.
[ ] 5. On customer's phone, open Lumina app. Sign in to customer's account
       (or assist them with the first sign-in if it's their first time).
[ ] 6. App should auto-discover the controller via mDNS within 1-2 minutes.
       If it doesn't, manually add via System → System Management →
       Controllers → Add Controller, and enter the IP shown in the router admin
       (or the .local hostname you set in Section 2.2 step 4).
[ ] 7. App should also discover the bridge via Firestore /bridge_registry.
       Tap Pair to complete the bridge pairing handshake (skip if Option B
       pre-pairing was used).
[ ] 8. In Lumina app: System → System Management → Remote Access → tap
       "Detect Home Network".
       Grant Location permission when prompted. App captures the customer's
       SSID hash for connectivity classification.
[ ] 9. Test on-wifi: tap power on/off, change brightness, apply a pattern.
       Lights should respond in under a second.
[ ] 10. Test off-wifi: enable phone hotspot, disconnect customer's phone
        from home WiFi, connect to hotspot, repeat power on/off and apply.
        Lights should respond in ~5-10 seconds through the cloud relay.
[ ] 11. Build the customer's nightly schedule with CLOCK times, then tap
        Sync on the Schedule tab and confirm the green "Schedules synced
        to controller" message. Schedules are a LAN-only write - this
        cannot be finished after you leave.
[ ] 12. Configure Now Playing label and any quick patterns the customer
        wants pre-set.
[ ] 13. Customer signs install acceptance form.
```

If any step fails, refer to Section 6.

Section 6 — Troubleshooting

Controller doesn't connect to customer WiFi at install site

Symptom: After plug-in, controller stays in AP mode (`WLED-AP-XXXX` still visible).

Cause + fix:

- **Wrong SSID or password**: re-enter via the AP captive portal. Read carefully — `0` vs `O`, `1` vs `l` vs `I`, capital vs lowercase.
- **5 GHz only network**: WLED supports 2.4 GHz only. Customer needs to ensure their router broadcasts 2.4 GHz, or split their network into a separate 2.4 GHz SSID for IoT.
- **Hidden SSID**: WLED supports hidden networks but requires manual entry (not from scan list). Re-enter via captive portal manually.
- **MAC filtering on customer's router**: have the customer add the controller's MAC to the allow list. MAC is on the controller's label or visible in WLED's Info page.

Bridge stays in `Lumina-XXXX` AP mode at install site

Symptom: Bridge's AP name still appears in phone WiFi list 5+ minutes after power-on at the install site.

Cause + fix: Same checks as the controller's WiFi issue — usually a typo'd SSID/password during pre-config. Connect to the bridge AP and re-enter creds.

Lumina app doesn't see the bridge in pairing flow

Symptom: Open app → Remote Access settings → "no bridges found".

Diagnostic:

- Verify bridge is on customer WiFi (it's not in AP mode anymore).
- In Firebase Console (if you have access) check `/bridge_registry/<deviceId>` exists with `lastSeen` updated within the last minute. If the doc doesn't exist, the bridge hasn't reached Firestore — check WiFi connectivity from the bridge.
- Confirm the customer is signed in to the correct Lumina account in the app.

App says "Could not read WiFi name" when tapping Detect Home Network on iOS

Cause: An older build had an iOS configuration gap. Confirm the customer is on the current released build; if they're on an older TestFlight install, update first. Location permission is required on both platforms to read the Wi-Fi network name.

Power tap takes 30+ seconds off-wifi

Cause: Bridge firmware too old (Item #76 fix not yet flashed). The current firmware (`POLL_INTERVAL_MS = 1000` , v1.2) should give ~5-10s remote latency. If higher, the bridge needs re-flashing (Appendix A).

Lights flash briefly on power-on then go dark

Cause: WLED's "Turn LEDs on after power up" was checked. Toggle OFF via Config → LED Preferences. Or accept the flash if customer prefers immediate-on behavior.

Appendix A — Flashing bridge firmware (if not pre-flashed)

If you receive raw/blank ESP32 boards, flash them with the Lumina bridge firmware before pre-config.

One-time setup on the flashing workstation:

```
# Install PlatformIO if not already installed
# Typically at C:\Users\<user>\.platformio\penv\Scripts\pio.exe on Windows
```

For each bridge:

```
# 1. Plug bridge into USB.
# 2. Identify the COM port:
#    PowerShell:
#      Get-CimInstance Win32_PnPEntity |
#        Where-Object { $_.Name -match 'COM\d+' -and
#                       ($_.DeviceID -match 'VID_1A86&PID_752[23]' -or
#                        $_.DeviceID -match 'VID_10C4&PID_EA60') } |
#        Select-Object Name, DeviceID | Format-List
# 3. Note the COMxx number for the chip just plugged in.

# 4. From the repo root:
cd esp32-bridge

# 5. Erase NVS (flash):
/c/Users/<user>/.platformio/penv/Scripts/pio.exe run --target erase --upload-port COMxx

# 6. Flash firmware:
/c/Users/<user>/.platformio/penv/Scripts/pio.exe run --target upload --upload-port COMxx
```

The serial output during erase prints the MAC address — **record this on the bridge label** as `deviceId` (MAC with colons stripped, e.g. `D4E9F4FA8E78`). It's the doc ID under `/bridge_registry` and the install crew uses it for visual confirmation in the app.

Appendix B — Default LED hardware settings (Lumina baseline)

When in doubt, use these defaults:

Setting	Value	Notes
LED type	SK6812 RGBW (30)	Lumina default; also use for WS2814
Color order	GRB (1)	All Lumina-supplied strips
Color depth	24-bit	RGBW handled via 4th channel
Use gamma correction	✓ ON	"Use Gamma correction for color"
Max current	60 mA × LED count	Cap at PSU rating
Default brightness	200/255 (~78%)	Conservative; customer can raise
Apply preset on boot	0 (none)	App controls state
Turn LEDs on at boot	OFF	Avoid surprise blast

Pin guidance: On the QuinLED Dig-Octa Brainboard-32-8L the channel-to-GPIO map is fixed and validated by QuinLED — use the table below as-is, including LED 1=GPIO 0, LED 2=GPIO 1, LED 4=GPIO 3 (strapping/UART pins that are safe on this board). If you're wiring a **generic** ESP32 board instead, GPIO 2, 4, 5, 13–19, 21–23, 25–27, 32–33 are reliable for LED data; UART0 (GPIO 1, 3) and GPIO 12 require care on bare ESP32s.

Channel-to-pin map (QuinLED Dig-Octa Brainboard-32-8L):

Channel	GPIO	Notes
LED 1	0	Strapping pin — works on Dig-Octa
LED 2	1	UART TX0 — UART logging unavailable when LED 2 is in use
LED 3	2	Strapping pin
LED 4	3	UART RX0
LED 5	4	General-purpose, reliable
LED 6	5	General-purpose, reliable
LED 7	13	General-purpose, reliable
LED 8	21	Shares I2C SCL bus — avoid pairing with I2C peripherals

---

Appendix C — Customer takeaway sheet

Print and hand to the customer at install completion:

```
Welcome to Lumina!

Your system at a glance
  Controller:    lumina-<customer>.local
  Lights:        <X> total LEDs across <N> channels
  Bridge:        deviceId <MAC>
  WiFi:          <SSID> (2.4 GHz)
  Installed by:  <dealer staff>  on  <date>

Daily use
  - Open the Lumina app on your phone (signed in to your account)
  - Tap the power circle to turn lights on/off
  - Slide the brightness bar to adjust
  - Tap a pattern card to apply a look
  - Lumina chat (the center icon) for natural-language requests

Away from home
  - Off your home WiFi, the app routes commands through the bridge
  - There may be a small delay (~5-10 seconds) for remote commands
  - Bridge stays on at all times — please don't unplug

Trouble?
  - Power-cycle the bridge (unplug, wait 10 seconds, plug back in)
  - Power-cycle the controller (cut and restore the 12V supply)
  - If neither helps, call us at <dealer phone>

Warranty + support contact: <dealer info>
```

Change log

- **2026-07-17**: Added §2.0 — **pin the WLED version** (SKIKBILY 4-channel → 0.15.1) with a post-flash `curl /json/info` verify. Field finding: WLED 0.15.4 (quinled.info "latest") has a periodic both-core stall regression on this board (~17s period, ~30% duty) that 0.15.1 does not; only the version differed across two same-variant units. Corrected §2.5 — AudioReactive is overhead to disable, but was wrongly blamed for the freeze; the freeze is the version regression, reproduced with AudioReactive off.
- **2026-05-13**: Initial publication. Reflects firmware v1.2 (`POLL_INTERVAL_MS=1000` , item #76 fix), iOS Item #75 Podfile fix, and Now Playing fix bundle items #88a–e.